Vinfotech

COMPUTER VISION TEAM · HIRING

CV Engineer Take-Home Case Study

Two problems drawn from the work our Computer Vision team does every day.

Overview

You have 3 days to complete two case studies and submit them together.

	Case Study 1	Case Study 2
Problem	Assembly verification	Traffic intersection analytics
Input	Images we provide	A live traffic camera stream
Output	OK / NOT OK per image, with missing parts marked	Counts, turning movements, queues and wait times

There is no single correct solution. We want to see how you think, how you structure a problem, and how you handle ambiguity. A clear, working pipeline with honest limitations beats an ambitious one that does not run.

You may use any tools, including AI assistants. In the interview you will be asked to explain your code, defend your decisions and run it live.

Case Study 1: Assembly verification

Build a program that checks a gear assembly and decides whether every required component is present and in its correct place.

The setup

- A fixed camera looks straight down at each assembly on an inspection station.
- From one image to the next, the assembly can shift a few millimetres and rotate by up to ±5°.

- Gears can sit at any rotation, and lighting varies slightly.

Required components

Component	Quantity
Base plate	1
Corner mounting bolts with washers	4
Large brass output gear	1
Washer and nut on the output gear	1
Middle (idler) gear	1
Circlip on the idler gear	1
Compound gear (upper gear visible)	1
Circlip on the compound gear	1
Bearing blocks	2
Screws on the bearing blocks	4 (2 per block)
Motor on its mounting plate	1
Screws on the motor mounting plate	2
Pinion gear on the motor	1
Cable connector plugged into the motor socket	1

What we provide

- `reference_good/` : 3 images of correct assemblies. Use these to build your program.
- `test_images/` : 6 unlabelled images, a mix of good and faulty assemblies. Use these to test your program.

We provide no labelled examples of faults. Your program must catch any missing or misplaced component, not only the faults that appear in the test images.

What your program must do

1. Run from one command on a single image, e.g. `python inspect.py --image path/to/image.png`.
2. Print a verdict: **OK** or **NOT OK**.
3. If NOT OK, list each missing or misplaced component by name.
4. Save an annotated image: each checked component marked, green if present, red if missing or misplaced.
5. Also run on a whole folder and print a summary table.
6. Run on a normal laptop CPU in a few seconds per image. No GPU required.

In the interview

We will give you new images you have not seen, both good and faulty. You will run your program on them live, and we will discuss the results with you.

Case Study 2: Traffic intersection analytics

Build a pipeline that measures how traffic moves through a signalised 4-way intersection, using a live traffic camera.

Input

- Live stream: traffic camera. Assume the camera is fixed.
- Pick your own **10–15 minute window in daylight**. It must cover at least **3 full signal cycles**.
- Record that window and submit the clip with your results. Everyone works on different footage, so we check your numbers against your own clip.
- You choose the frame rate. Explain why you chose it.

Definitions

Treat the top of the frame as North. The intersection has four **arms**: North, South, East and West.

- **Entry arm:** the arm a vehicle drives in from.
- **Turning movement:** the pair of arms a vehicle enters and exits by, e.g. South → East is a right turn.
- **Queue length:** the number of vehicles stopped and waiting on an arm at a given moment.
- **Wait time:** how long a vehicle waits, from when it stops in the queue to when it crosses the stop line.

If you find a definition ambiguous, decide how you will handle it and state it as an assumption.

Required outputs

1. **Count by entry arm:** total vehicles entering from each arm.
2. **Turning counts:** a 4 × 4 table of entry arm × exit arm.
3. **Queues and waiting:** for each arm, the maximum queue length, plus the average and maximum wait time.
4. **Moving traffic only:** parked vehicles and vehicles that never pass through the intersection must not be counted.

Tracking is required: without it, stopped and turning vehicles cannot be counted correctly.

Bonus

1. **Speed estimation:** estimate the speed of each vehicle in km/h. Explain how you converted pixels to real distance and how accurate you think it is.

Expected output

- An annotated video of your window showing bounding boxes, track IDs, the arm zones and running counts.

- A results file (CSV or JSON) and a printed summary.

Allowed tools

Python, OpenCV, and pretrained detection and tracking models (e.g. Ultralytics YOLO, ByteTrack). No model training is required.

What to submit

Submit one Git repository link and a zip of the same code. Each case study needs the four items below.

1. Assumptions

List every assumption you made, clearly and by name. For example: "A vehicle stopped for more than 3 seconds within 30 m of the stop line counts as queued." Unstated assumptions count against you; stated ones that turn out wrong do not.

2. Approach write-up (200 words)

For each case study, write about **200 words** explaining your approach and how you implemented it: what you built, why you chose it, and where it breaks. Keep it within 200 words. We read this to judge how clearly you can explain your own work.

3. Outputs

- **Case Study 1:** an annotated image for every test image, plus the summary table.
- **Case Study 2:** the recorded clip, the annotated video, and the results file.
- Large files can be shared as a Google Drive or similar link.

4. README

Include setup steps, the exact command to run each case study, your assumptions, known limitations, and what you would improve with more time.

What happens next

If your submission is shortlisted, the next round is a discussion of your work. You will walk us through your approach, run Case Study 1 live on new images, and may be asked to make a small change to your code.